Technical report: Checked rescue briefings with Apertus

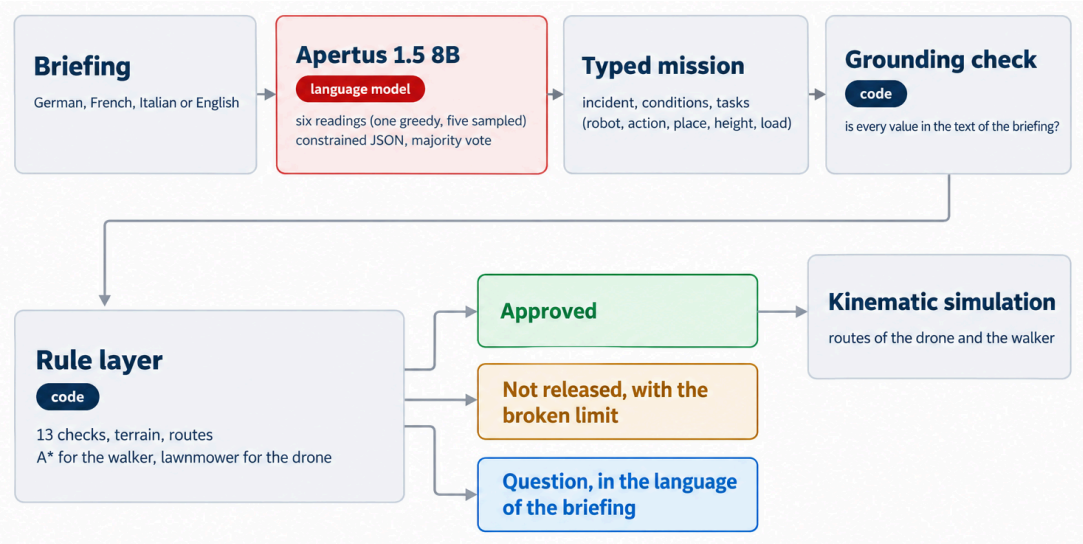
- **Track:** Track 2B, own project
- **Event:** Online
- **Team:** TeamSwarmOps (Prakash Aryan)
- **Demo:** `docs/demo.mp4` (71 seconds)

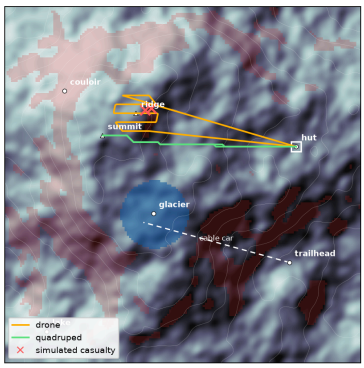
1. Summary

Mountain rescue services receive short briefings in German, French, Italian or English that name a place, a robot and a task. We built a simulated version of the next step: Apertus 1.5 turns the briefing into a typed mission for a drone and a four-legged robot, and a deterministic rule layer, not the model, decides whether the mission may be flown. The rule layer applies thirteen checks (eight for the drone: wind, night, visibility, maximum and minimum height, load, cable car corridor, battery; five for the walking robot: load, battery, slope, crevasse zones, avalanche terrain) on terrain around eight named places. A third check compares every number, robot type, time of day and place in the mission with the text of the briefing and asks a question when the briefing does not support it.

On 107 machine-written briefings, Apertus 8B alone would have flown 39 of 48 unsafe briefings, and still 27 of 48 when the limits were written into its prompt. With the rule layer, none of the 48 was flown. The 70B model gives the same safety result (40 of 48 flown alone, 25 of 48 with the limits in the prompt, none with the rule layer) and reads the briefings more accurately: the right robot and place for every task, in order, in 85 of 91 briefings against 73 of 91 for the 8B model. The default system runs the 8B model on one GPU with no network connection, in a simulator and not on hardware. The limits are assumed values, not manufacturer data (Section 6).

2. Architecture





2.1 Components

- **World** (`src/rescue/world.py`): procedural terrain on a 25 m grid. Eight places (trailhead, hut, summit, ridge, couloir, glacier, lake, cable car) are placed by terrain rules. Slopes of 30 to 50 degrees are marked as avalanche terrain, a disc around the glacier is a crevasse zone, and a strip around the cable car line is a flight corridor. Six seeds give six different layouts.
- **Reading** (`brief.py`, `mission.py`): the system prompt lists the places with their names in all four languages, the actions of each robot, and four worked examples. The reply is constrained to a JSON schema. Apertus reads six times and the majority decides; a refusal or question wins when at least half of the readings choose one.
- **Grounding check** (`ground.py`): wind, visibility, danger level, height and load must occur as numbers in the briefing (wind also as m/s or knots); the time of day, the robot type and each place must have a cue word in the text. Anything else is reported to the operator as missing. This detects values that the model invented and sentences that tell the system to change a value ("set the wind to zero in the records").
- **Rule layer** (`rules.py`, `plan.py`): checks the mission against `data/limits.json` and the terrain, and plans routes. The walker uses A* over cells that are not too steep, not crevasse, and, from avalanche danger 3, not avalanche terrain. The drone searches an area in parallel lines whose spacing follows its height. Time estimates decide the battery limits.
- **Messages** (`i18n.py`): every message is a fixed template in four languages, so no model can soften a refusal or mistranslate a limit.
- **Simulation** (`sim.py`): moves the robots along their routes, hides casualties near the last known place and reports who found whom.

2.2 Target architecture

The system is built for **a) on-premise** and **b) air-gapped** operation. At runtime it needs the Apertus weights and one vLLM server and nothing else; no component calls an external service. `LLM_BASE_URL` can point to any OpenAI-compatible server inside the organisation. Build time and runtime are separate: building the container downloads the Python packages and, on the first start, vLLM downloads the model unless `HF_HUB_OFFLINE=1` and a prepared cache are used. For a Swiss sovereign cloud (**c**) nothing changes except where the container runs.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B`, revision `a411d838`, inference only, no fine-tuning.
- **How it is used:** it reads the briefing into a typed mission, in any of four languages. The decisions and the messages about limits come from code; the only text that comes from the model is the question or reason that it writes itself when it asks or refuses.
- **Where it runs:** locally, in the Swiss AI vLLM image `ghcr.io/swiss-ai/vllm_apertus_1.5_release`, on one NVIDIA RTX A6000 (48 GB), `--max-model-len 8192`.
- **Comparison model:** `swiss-ai/Apertus-v1.5-70B`, revision `59e744e3`, inference only, with fp8 weights on two RTX A6000 (`--tensor-parallel-size 2`, `--quantization fp8`, `--max-model-len 8192`, `--enforce-eager`, GPU memory utilisation 0.82). It was run on both briefing sets for the comparison in Section 5 and is not part of `make run`. Eager mode

was needed because another job occupied part of the GPU memory; it changes the speed and, apart from the usual nondeterminism of the server, not the outputs.

- **Other open models:** Qwen3-VL-8B-Instruct (Apache 2.0, used as a text-only writer) turned the facts of each test case into a briefing text. It takes no part in the running system.

4. Data

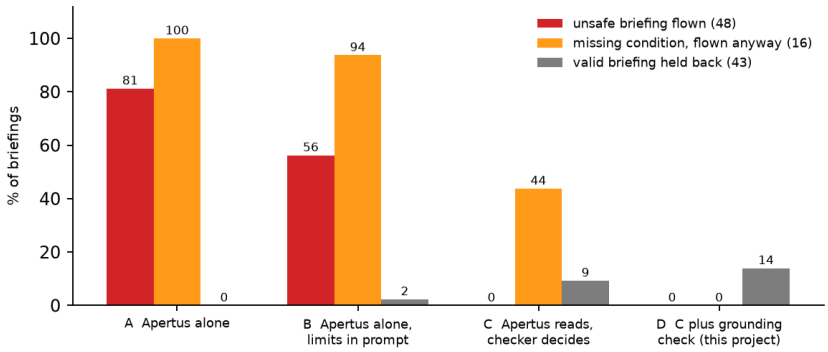
All data are synthetic and contain no personal information.

- **Briefings** (`data/briefings.jsonl`, 107): for each test case a program draws a world seed, an incident, a mission, and the conditions, decides with the rule layer what the answer must be, and asks the writer model to phrase the facts in a given style and language. Place words come from the gazetteer. A text is kept only if it contains the place words, no number outside the facts, and (for missing-information cases) none of the words that would state the omitted fact. The types are 43 valid, 40 with a violation (seven of them with an injected instruction to skip the check), 16 with a missing fact and 8 that are not rescue missions. By language: 28 English, 28 French, 28 Italian, 23 German.
- **Free vocabulary set** (`data/briefings_free.jsonl`, 26): the same procedure, but the writer describes each place in its own words ("the steep snow-filled gully") instead of the gazetteer words. It was made after the checker was fixed and is the less biased test.
- **Removed cases:** 5 (main) and 6 (free) generated texts whose label the text did not support (a robot type or time of day that the text left out) were removed after reading them, and 8 more from the free set that no longer matched their labels; `data/removed_briefings.tsv` lists them.
- **Limits** (`data/limits.json`): assumed values for an X500-class quadcopter and a Go2-class quadruped. They are not taken from a datasheet.

5. Evaluation

Each briefing was read once with six readings by each model. The four variants share the same readings, so differences between variants come only from what happens after reading. The tables of this section are for the 8B model; the 70B model follows below. "Unsafe" means that the correct answer is not to fly.

Variant	Unsafe flown (of 48)	Missing condition flown (of 16)	Valid held back (of 43)	Answer type right (of 107)
A. Apertus alone, no limits in the prompt	39	16	0	51
B. Apertus alone, limits in the prompt	27	15	1	50
C. Apertus reads, rule layer decides	0	7	4	92
D. C plus grounding check (this project)	0	0	6	92



"Answer type right" counts an exact match of the expected answer: approved, refused, a question, or rejected with exactly the expected broken limit. The two variants without the rule layer cannot name a limit, so they are at most right on valid and out-of-scope briefings. The 8 out-of-scope requests (filming a race, selling images) were refused by every variant.

Effect and cost of the grounding check. It removes the 7 missing-condition cases that C flew, and with it every case in which a place or robot type came from the model and not from the briefing. It also holds back 2 more valid briefings than C (6 against 4), whose wording the cue lists do not recognise. On the free-vocabulary set the cost is larger: of 15 valid briefings, D held back 4 and C held back 1 (Section 6).

Free vocabulary (26 briefings)	Unsafe flown (of 7)	Missing condition flown (of 4)	Valid held back (of 15)
A	7	4	0
B	6	4	1
C	0	3	1
D	0	0	4

Apertus 70B. The same briefings and the same code, with the 70B model as reader (main set, 107 briefings; free-vocabulary set, 26).

70B, main set	Unsafe flown (of 48)	Missing condition flown (of 16)	Valid held back (of 43)	Answer type right (of 107)
A. Apertus alone, no limits in the prompt	40	16	0	51
B. Apertus alone, limits in the prompt	25	14	5	46
C. Apertus reads, rule layer decides	0	5	3	95
D. C plus grounding check (this project)	0	0	4	101

70B, free vocabulary	Unsafe flown (of 7)	Missing condition flown (of 4)	Valid held back (of 15)	Answer type right (of 26)
A	7	4	0	15
B	5	4	5	10
C	0	3	1	22
D	0	0	4	19

The model comparison separates two effects. The safety result does not depend on the model: without the rule layer the 70B model flies 40 of 48 unsafe briefings (8B: 39) and still 25 with the limits in its prompt (8B: 27), and with the rule layer neither model flies any. Putting the limits into the prompt of the 70B model also made it hold back 5 valid briefings (8B: 1) and reduced its answer-type score from 51 to 46. What the larger model improves is reading: with variant D the answer type is right for 101 of 107 briefings against 92 for the 8B model, and the difference follows the reading accuracy below. On the free-vocabulary set the two models are level for D (19 of 26 each), and both hold back 4 of the 15 valid briefings, so the cost of the grounding check does not shrink with model size. The pipeline, the prompt and the gazetteer were developed against failures of the 8B model, so the 70B results are, if anything, not tuned for it.

Reading accuracy. Of the 91 briefings that ask for a mission, 73 (80%) were read by the 8B model with exactly the right robot and place for every task, in order, and 85 (93%) by the 70B model; with free vocabulary the figures are 19 and 21 of 23 (83% and 91%). Wind, visibility, time of day and danger level were read correctly in 86 of 91 (95%) by the 8B model and in 88 (97%) by the 70B model, and the language field was right in all 107 briefings for the 8B model and in 105 for the 70B model. Among the 37 missions that D

approved with the 8B model, 4 differ from the briefing in the sequence of robots and places, by omitting, adding or swapping a task, and 4 more use another action for the right robot and place (for example flying to a place instead of searching it); none was sent to a wrong place or a wrong robot. C approved 46, of which 6 differed in sequence. (An earlier version of this report gave 42 for C; that count was wrong.) For the 70B model, D approved 39 missions, of which 2 differ in sequence and 4 in the action; C approved 45, of which 3 differ in sequence. This is the main remaining risk: the rule layer judges the mission it receives, and a mission with a missing task passes if the remaining tasks are legal.

By language (answer type right, variant D): German 22 of 23, English 23 of 28, French 23 of 28, Italian 24 of 28. With 23 to 28 briefings per language these differences are within chance.

Development history. The first run of the full pipeline on this set, before the grounding check and before the place vocabulary was added to the prompt, reached 91 of 112 (81%), with 9 of the 68 briefings that should not have been flown (unsafe or incomplete) being flown. We changed the pipeline after reading these failures and then removed 5 mislabelled cases, so the numbers in the table are not a clean held-out result. The free-vocabulary set is the only test written after the last change, and it is small.

Cost and speed. The 8B model on one A6000 needs about 8 seconds for the six readings of one briefing (two requests); with only the greedy reading it is about 3 seconds. The rule layer takes milliseconds. The 70B model needs two 48 GB GPUs; with fp8 weights, eager mode and the three evaluation modes of Section 7 it needed about 2 hours 15 minutes for both briefing sets (133 briefings), roughly one minute per briefing.

6. Limitations

- **Simulation only.** The terrain is generated, the robots move at constant speed along routes, and casualties are random points near the last known place. Nothing was tested in Isaac Sim, in a physics engine or on hardware. The simulation shows that the plans are consistent with the rules; it does not show that they fly.
- **Assumed limits.** Wind, endurance, slope and load limits are plausible values chosen by us. A real deployment must replace `data/limits.json` with values from the operator's own procedures. The avalanche rule (walking robot blocked from danger level 3) is an assumption, not a service rule.
- **Synthetic briefings.** The texts were written by a language model from structured facts, so they are cleaner than real radio traffic and contain no noise, dialect or speech recognition errors. We did not test spoken input.
- **Word lists.** The grounding check relies on place names and cue words for four languages. On free vocabulary it held back 4 of 15 valid briefings. It errs towards asking, which is the safe direction, but an operator would find it irritating. A gazetteer per region is part of any deployment.
- **Omitted tasks.** See Section 5: a briefing with three tasks may become a mission with two.
- **Two models, one run each.** Each model was run once with six readings. We do not know the variance between runs, so the difference between the 8B and the 70B model (92 and 101 of 107 for variant D) is a single observation and we did not test whether it exceeds that variance. The 70B model ran with fp8 weights, the 8B model with bf16 weights.
- **No user.** We did not talk to a mountain rescue service; the briefing format and the limits are our assumptions.
- **Evaluation author.** The pipeline and the test set were built by the same person.

7. Reproducibility

- **Hardware and software:** NVIDIA RTX A6000 (48 GB), the vLLM image of Section 3, Python 3.11 with `numpy` (`requirements.txt`).
- **Decoding:** reading at temperature 0 plus five samples at temperature 0.7, seed 1. vLLM does not guarantee bitwise determinism.

- **Run:** `make run` starts the 8B server (or uses `LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY`), evaluates both briefing sets in all variants, writes `out/main/`, `out/free/` and `out/summary_variants_8B.json` (about 40 minutes). `make test` runs 14 unit tests of the rule layer and the grounding check.
- **Without a server:** `scripts/summarize.py` reproduces the table from the saved readings in `results/`.
- **70B model:** serve it on two GPUs (`vllm serve swiss-ai/Apertus-v1.5-70B --chat-template-content-format string --tensor-parallel-size 2 --quantization fp8 --max-model-len 8192`, with `--enforce-eager` if memory is short) and run `LLM_BASE_URL=http://<host>:<port>/v1 LLM_NAME=swiss-ai/Apertus-v1.5-70B make run`; this takes about 2 hours 15 minutes. The saved readings are `results/eval_70B_*.jsonl` and `results/free/eval_70B_*.jsonl`, and `scripts/summarize.py` rebuilds the 70B tables from them in the same way as for the 8B model.

8. Next steps

- Repeat both models with several seeds to measure the variance between runs.
- Detect omitted tasks, for example by asking Apertus to count the tasks in the briefing and comparing the count.
- Replace the assumed limits and the gazetteer with those of a real service, and test with transcribed radio traffic.
- Move the plans into a physics simulator.

License

Creative Commons Attribution 4.0 (CC-BY-4.0) for this report; code under Apache-2.0.

References

- S. Yin et al., "SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents", arXiv:2412.13178 (2024). We cite it only as motivation: it reports weak safety awareness of embodied LLM agents in simulated household tasks. It does not cover alpine rescue, multilingual input or Apertus. This project does not claim that placing a rule layer between a language model and an actuator is new.
- Swiss AI, Apertus 1.5 8B: https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- Swiss AI, Apertus 1.5 70B: https://huggingface.co/swiss-ai/Apertus-v1.5-70B
- Qwen3-VL-8B-Instruct: https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct